

Builder Design Pattern

Why

- ① Class that has a lot of attributes

class Student {

name

age

pcp

batch

id

gender

}

① Create obj of Student

② Set value of all of its attributes

⇒ {
Student s = new Student();
s.setAge(12);
s.setName(Aman);
s.setGender(Male);
}

② We want to validate the value of attributes

{ Can involve multiple attributes }

| if age > 18 then pcp > 70

⇒ Will we be able to do this using

default constructor and letters.

Student st = new Student()
| St. setAge(18); | St. setPsp(→)
~~St. setPsp(→);~~

Problem Statement

- ① Class with a lot of attributes
a.) we wanted to validate the attr of the class.

Solⁿ → Create obj using constructor
→ validate in class

- 1.) Not going to work
- 2.) Won't work if class is immutable.

Problem Statement

- ① Class with a lot of attr ←
- ② Validate the value of attr of that class ←
- ③ Class may be immutable *
- ④ Validations may involve multiple attr *

Solⁿ 1

Class Student {

Student() { }

Set Name (—) { - }

Set Age (—) { - }

Set Gender (—) { - }

① Want
handle

multiple attr
validation

② Want make

if class is
immutable

Once you have created
obj of the class you
shouldn't be required
to modify it.

↓
Not Setters

Ques 2

Validate using constructor

Class Student {

String name;
int age;
String gender;
double psp;

String batch;
String in Name;
String email;

⇒ Student (String name, int age, String gender, double psp) {
if (name == null or age < 18)
throw Exception ()
if (age > 18 and psp < 45)
throw Exception ()
}
// Set attrs

Client {

⇒ Student st = new Student ("Naman", 24, "Male", 80.0);

new Student ("Male", 80, "Naman", 24)

}

Problems

⇒

- ① Client is going to have difficulty creating obj.
- ② Prone to errors.

⇒

- ③ We haven't actually validated before the obj was created.

- ④ Not easily understandable.

- ⑤ lot of nulls in constructor

Client {

```
Student st = new Student (Naman, Male,  
                        80, 24, null,  
                        null, Anon, null)
```

}



```

class Student {
    int age;
    String name;
    double pop;
    String gender;
    String university;

```

```

    Student (age, name) {
        // validate
        set;
    }

```

```

    }

    Student (String name, String batch) {
        // validate
        set;
    }

```

Error

```

    Student (String name, String uniName) {
        // validate
        set;
    }

```

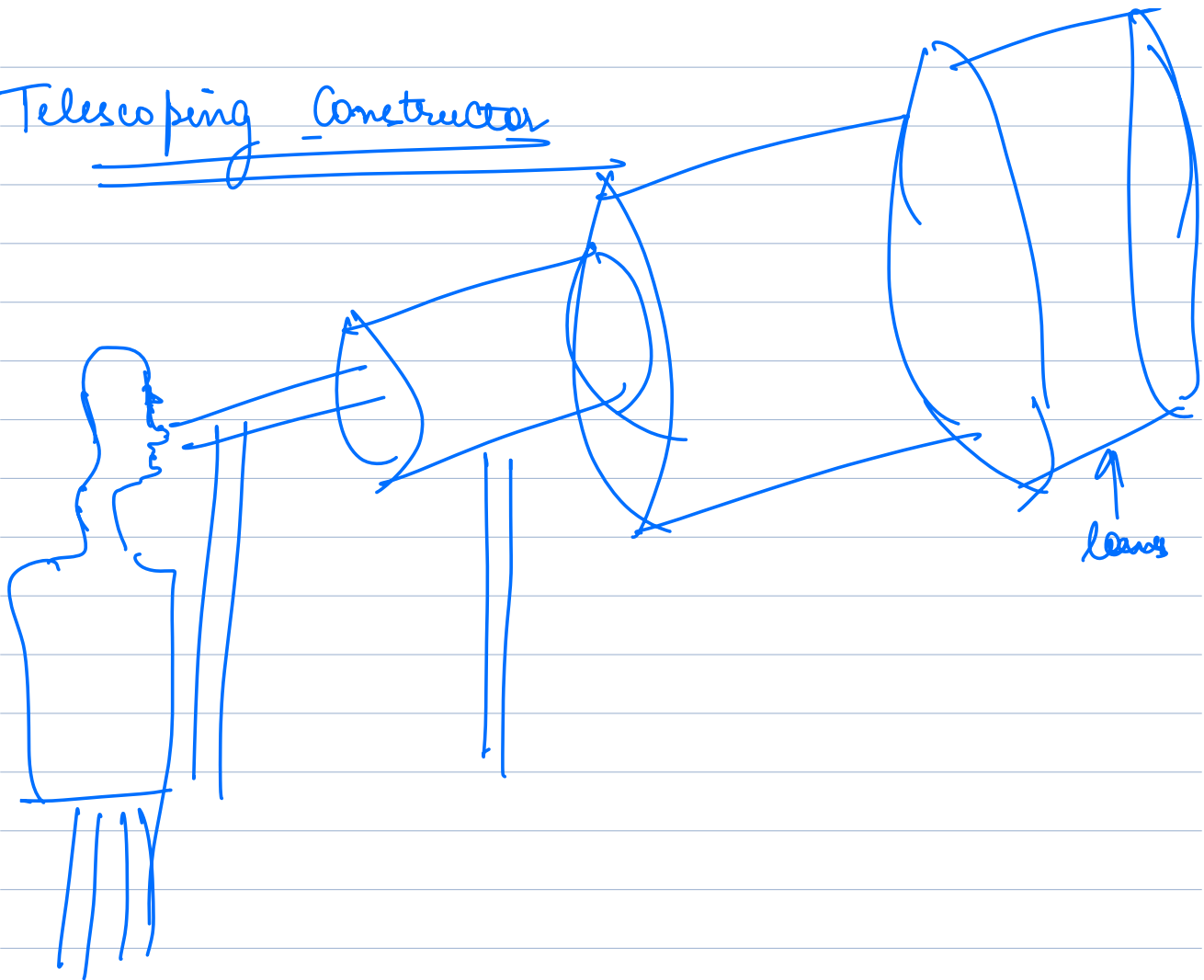
Problems

→ Too many constructors (2^n constructors)

→ Not possible to create all combⁿ

→ Repeated code (DAY)

Telescoping Constructor



```
class Student {  
    String name;  
    int age;  
    String gender;  
    double pop;  
    String uniName;  
}
```

```
Student (name) { }  
Student (name, age) {  
    this (name);  
    // or this (age);  
}
```

validation of both name and age
~~Set Name~~
 Set Age

obj

```
Student (name, age, batch Name) {
  this (name, age) =
  // validate name, age, batch name.
  Set batch Name.
}
```

Problems

→ Still first 2 problem
 → which cons will you even call.
 ↳ still some dupⁿ will remain

① Only place where I can do something
 is constructor

② But in cons

① Explosion (⇒ My solⁿ should have only 1 cons)

② Readability (⇒ Understanding what is

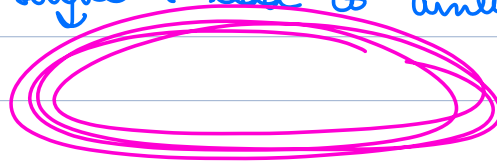
what is not easy).

class Student {

→ not need me to send null
→ Multiple values.

→ each value should have a name,
shouldn't lead to ambiguity

Student {



→ 1 thing

→ have multiple values

→ each value can be
recognized by a name

→ no need to pass if doesn't
exist

}

}

class Student {

String // Object

Student (Map < , > arguments) {
String name = args.get("name")
double pep = args.get("pep")

// validations

}

}

```

Map < String, Object > args = new HashMap<>();
args.put("name", "Arman");
args.put("age", 13);

```

⇒ A good codebase should be able to detect as many issues as possible at compile time rather than run time.

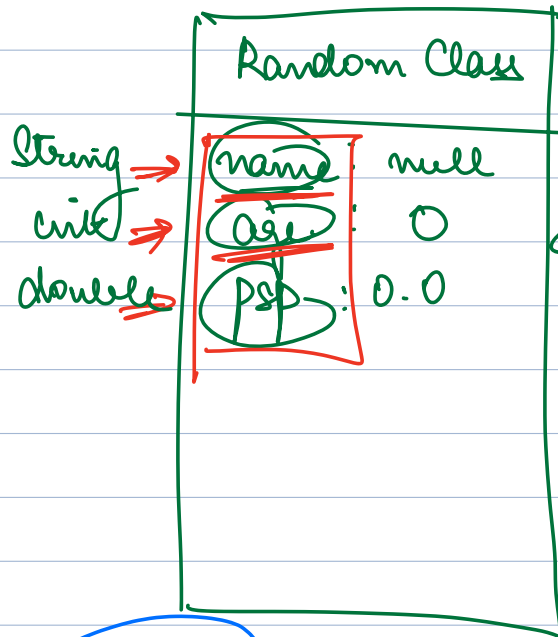
Problems

- ① Types in key
- ② Data type that is passed might be wrong.

We need something:

- ① have multiple values
- ② each value should have a name.
- ③ default value / empty if something is not passed.
- ④ each value should have a name that is fixed (types shouldn't be there / only fixed set of keys)
- ⑤ data type of each key must be fixed / specific.

Problem in Map



Ac. name = "nana"

· XYZ = 123

Production

```
Class Student {
    String name;
    int age;
    String gender;
}
```

Staging env

```
Class Builder {
    String name;
    int age;
    String gender;
    boolean validate() {
        //
        //
        //
    }
}
```

```
⇒ Student (Builder b) {
    // validate attrs in builder.
    this.age = b.age;
    this.name = b.name;
    this.gender = b.gender;
}
```

Why okay:

- ① no need of immutability
- ② no req of validation

Cons with
Setters will
work

Builder b = new Builder();
 {
 b. set Name (Naman)
 b. set Age (24)
 b. set Gender (Male)
 b. set Psp (87)
 }
 if (! b. validate())
 throw Exception(____);
 }

95%
done

① Student s = new Student(u);

y/s
 Student s = Student. create()
 {
 set Name (____)
 set Age (____)
 set Gender (____)
 build >
 }

Break till 10:27 PM